ECSE 222, Digital Logic

VHDL 6:

Finite State Machines in VHDL

Fall 2023
University of McGill

Presented by:

Nara Yun (Student #: 261115268)

Karen Chen Lai (Student #: 261107674)

Instructors: Prof. Boris Vaisband

Teaching Assistant: Maninder Bir Singh Gulshan

Date: December 5th, 2023

Executive Summary

In this assignment, the objective was to design and implement a sequence detector circuit using VHDL, targeting an Altera DE1-SoC FPGA board. The sequence detector is required to identify two distinct bit patterns, '1011' and '0010', in an input bit sequence. The implementation involves using Moore-type Finite State Machines (FSMs) with asynchronous reset and enable signals. Two FSMs are employed, each dedicated to detecting one of the specified bit patterns.

The VHDL code for the sequence detector is organized into an entity named "firstname_lastname_FSM," where "Karen_Chen" represents the name of one of the students in the group. The entity has input ports for the sequence of bits (seq), enable signal (enable), reset signal (reset), clock signal (clk), and output signals (out_1 and out_2) indicating the detection of the respective bit patterns.

Following the sequence detector implementation, the assignment also requires the development of a sequence counter circuit. This circuit counts the occurrences of the specified bit patterns using the sequence detector and two 3-bit up-counters. The counters increment when their corresponding patterns are detected.

Questions

1. Why is it better to use two FSMs, rather than one, in the implementation of the sequence detector from Section 4?

Because while using 1 FSM in Section 4, it would be much more complicated to find patterns for “1011” and “0010” since it requires more states and can easily be messed up. In contrast, utilizing 2 FSMs can save much more running time since two FSMs can be carried out within similar time ranges and each of them have their own patterns and states in order to better follow the tracking progress.

2. Briefly explain your VHDL code implementation of all circuits.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.ALL;

entity Karen_Chen_FSM is
port (seq : in std_logic;
      enable : in std_logic;
      reset : in std_logic;
      clk : in std_logic;
      out_1 : out std_logic; -- generates 1 when the pattern "1011" is detected; otherwise 0.
      out_2 : out std_logic); -- generates 1 when the pattern "0010" is detected; otherwise 0.
end Karen_Chen_FSM;

architecture FSM of Karen_Chen_FSM is

   type stateOne is ( stateA, stateB, stateC, stateD, stateE); -- 1011 define type
   signal stateOneCurrent, stateOneNext: stateOne;

   type stateTwo is ( stateF, stateG, stateH, stateI, stateJ); -- 0010 define type
   signal stateTwoCurrent, stateTwoNext: stateTwo;

   begin

      process(clk,reset)
      begin

           if(reset='0') then -- reset if reset is 0
              stateOneCurrent <= stateA;
              stateTwoCurrent <= stateF;
           elsif(enable='0') then
              stateOneCurrent <= stateOneCurrent;
              stateTwoCurrent<=stateTwoCurrent;
           elsif(rising_edge(clk)) then
              stateOneCurrent <= stateOneNext;
              stateTwoCurrent <= stateTwoNext;
           end if;
      end process;

      process(stateOneCurrent,seq) -- 1011
      begin
         case (stateOneCurrent) is
            when stateA =>
               if (seq='1') then
                  stateOneNext <= stateB;
               else
                  stateOneNext <= stateA;
               end if;

            when stateB =>
               if (seq = '0') then
                  stateOneNext <= stateC;
               else
                  stateOneNext <= stateB;
               end if;

            when stateC =>
               if (seq = '1') then
                  stateOneNext <= stateD;
               else
                  stateOneNext <= stateA; -- go back to A
               end if;

            when stateD =>
               if (seq = '1') then
                  stateOneNext <= stateE;
               else
                  stateOneNext <= stateC;
               end if;

            when stateE =>
               if (seq = '1') then
                  stateOneNext <= stateB;
               else
                  stateOneNext <= stateC;
               end if;

            when others => null;

         end case;
      end process;

      process (stateTwoCurrent, seq) -- 0010
      begin
         case (stateTwoCurrent) is
            when stateF =>
               if (seq = '0') then
                  stateTwoNext <= stateG;
               else
                  stateTwoNext <= stateF;
               end if;

            when stateG =>
               if (seq = '0') then
                  stateTwoNext <= stateH;
               else
                  stateTwoNext <= stateF;
               end if;

            when stateH =>
               if (seq = '1') then
                  stateTwoNext <= stateI;
               else
                  stateTwoNext <= stateH;
               end if;

            when stateI =>
               if (seq = '0') then
                  stateTwoNext <= stateJ;
               else
                  stateTwoNext <= stateF;
               end if;

            when stateJ =>
               if (seq = '0') then
                  stateTwoNext <= stateH;
               else
                  stateTwoNext <= stateF;
               end if;

            when others => null;

         end case;
      end process;

      process(stateOneCurrent)
      begin
         case stateOneCurrent is
            when stateE =>
               out_1 <= '1';

            when others =>
               out_1 <= '0';
         end case;
      end process;

      process(stateTwoCurrent)
      begin
         case stateTwoCurrent is
            when stateJ =>
               out_2 <= '1';

            when others =>
               out_2 <= '0';
         end case;
      end process;


end FSM;
```

Figure 1. FSM behavioral VHDL description

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_FSM_tb is
--empty
end Karen_Chen_FSM_tb;

architecture tb of Karen_Chen_FSM_tb is
component Karen_Chen_FSM is
   port (   seq      : in std_logic ;
            enable   : in std_logic ;
            reset    : in std_logic ;
            clk      : in std_logic ;
            out_1    : out std_logic ;
            out_2    : out std_logic );
end component;

signal seq, enable, reset, clk, out_1, out_2: std_logic;
constant clock_period : time := 10 ns;

begin
   uut: Karen_Chen_FSM port map (seq, enable, reset, clk, out_1, out_2);

   clock_generation :process
   begin
        clk <= '1';
       wait for clock_period/2;
       clk <= '0';
       wait for clock_period/2;
   end process clock_generation;

   process
      begin
        enable <= '0';
        reset <= '0';
        seq <= '0';
        wait for 20 ns;

        enable <= '0';
        reset <= '1';
        seq <= '0';
        wait for 20 ns;

        enable <= '1';
        reset <= '0';
        seq <= '0';
        wait for 20 ns;

        enable <= '1';
        reset <= '1';
        seq <= '0';
        wait for 20 ns;

        enable <= '1'; ------
        reset <= '1';

        seq <= '0';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '0';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

        seq <= '1';
        wait for 10 ns;

    end process;
end;
```

Figure 2. FSM Testbench code

The Finite State Machine (FSM) for the Sequence Detector takes a sequence of bits (seq) as input along with an enable signal (enable), an asynchronous active-low reset signal (reset), and the clock signal (clk). The FSM produces two distinct outputs: out_1 is set to 1 when the pattern '1011' is detected and 0 otherwise, while out_2 is set to 1 when the pattern '0010' is detected and 0 otherwise. The implementation of this FSM involves the use of a Moore-type state machine, designed to efficiently detect the specified bit patterns. The states and transitions within the FSM are carefully crafted based on the state diagram, ensuring a systematic and controlled progression through states. Asynchronous reset and enable signals have been integrated into the FSM to manage state transitions and maintain controlled operation, enhancing the reliability and adaptability of the circuit.

```vhdl
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.NUMERIC_STD.all;

entity Karen_Chen_sequence_detector is
port (seq : in std_logic;
      enable : in std_logic;
      reset : in std_logic;
      clk : in std_logic;
      cnt_1 : out std_logic_vector(2 downto 0); -- counts the occurrence of the pattern "1011".
      cnt_2 : out std_logic_vector(2 downto 0)); -- counts the occurrence of the pattern "0010".
end Karen_Chen_sequence_detector;

architecture sequence_detector of Karen_Chen_sequence_detector is

   component Karen_Chen_FSM is
      port (seq : in std_logic;
               enable : in std_logic;
               reset : in std_logic;
               clk : in std_logic;
               out_1 : out std_logic; -- "1011"
               out_2 : out std_logic); -- "0010"
   end component;

   component Karen_Chen_counter is
      port (   enable   : in std_logic;
               reset    : in std_logic;
               clk      : in std_logic;
               count    : out std_logic_vector(2 downto 0));
   end component;

   signal count1,count2:std_logic;

begin

   FSM : Karen_Chen_FSM port map (seq,enable,reset,clk,count1,count2);

   counter1 : Karen_Chen_counter port map (count1,reset,clk,cnt_1);

   counter2 : Karen_Chen_counter port map (count2,reset,clk,cnt_2);


end sequence_detector;
```

Figure 3.Sequence Counter behavioral VHDL description

```vhdl
library ieee;
use ieee.std_logic_1164.all;
use ieee.numeric_std.all;

entity Karen_Chen_sequence_detector_tb is
-- empty
end Karen_Chen_sequence_detector_tb;

architecture tb of Karen_Chen_sequence_detector_tb is

    component Karen_Chen_sequence_detector is
    port (    seq      : in std_logic ;
              enable   : in std_logic ;
              reset    : in std_logic ;
              clk      : in std_logic ;
              cnt_1    : out std_logic_vector(2 downto 0) ; -- 1101
              cnt_2    : out std_logic_vector(2 downto 0) ); -- 0010
    end component;

    signal seq, enable, reset, clk: std_logic;
    signal cnt_1, cnt_2: std_logic_vector(2 downto 0) ;
    constant clock_period : time := 10 ns;

begin
    uut: Karen_Chen_sequence_detector port map (seq, enable, reset, clk, cnt_1, cnt_2);

    clock_generation :process
    begin
         clk <= '1';
        wait for clock_period/2;
         clk <= '0';
         wait for clock_period/2;
    end process clock_generation ;

    process
        begin
          --enable <= '0'; For testing if they are working well
          --reset <= '0';
          --seq <= '0';
          --wait for 20 ns;

          --enable <= '0';
          --reset <= '1';
          --seq <= '0';
          --wait for 20 ns;

          --enable <= '1';
          --reset <= '0';
          --seq <= '0';
          --wait for 20 ns;

          --enable <= '1';
          --reset <= '1';
          --seq <= '0';
          --wait for 20 ns;

          enable <= '1'; -- testing for counting
          reset <= '1';

          seq <= '0';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '0';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

          seq <= '1';
          wait for 10 ns;

    end process;
end;
```

Figure 4. Sequence Counter testbench code

The Sequence Counter operates on a sequence of bits (seq) provided as input, alongside an enable signal (enable), an asynchronous active-low reset signal (reset), and the

clock signal (clk). The output of this counter consists of two 3-bit vectors: cnt_1 represents the count of occurrences of the pattern '1011,' and cnt_2 signifies the count of occurrences of the pattern '0010.' The implementation strategy involves leveraging the previously designed sequence detector circuit to identify specific bit patterns and trigger increments in the counters accordingly. To achieve this, two 3-bit up-counters have been integrated into the design, each dedicated to tracking occurrences of one of the specified patterns. The implementation ensures accurate counter increments upon pattern detection, and careful handling of the reset and enable signals is incorporated to maintain the integrity of the counting process. This comprehensive approach ensures the reliable and precise tracking of occurrences for both specified bit patterns in the input sequence.

```
1   library IEEE;
2   use IEEE.STD_LOGIC_1164.ALL;
3   use IEEE.NUMERIC_STD.ALL;
4   entity Karen_Chen_wrapper is
5   Port  (   reset   : in std_logic ;
6             clk     : in std_logic ;
7             HEX0    : out std_logic_vector(6 downto 0);
8             HEX5    : out std_logic_vector(6 downto 0) );
9   end Karen_Chen_wrapper ;
10
11  architecture wrapper of Karen_Chen_wrapper is
12
13     component Karen_Chen_clock_divider is
14        port (   enable    : in std_logic;
15                 reset     : in std_logic;
16                 clk       : in std_logic;
17                 en_out    : out std_logic );
18     end component;
19
20     component ROM
21        port(clk : in std_logic;
22             reset : in std_logic;
23             data : out std_logic);
24     end component;
25
26     component Karen_Chen_sequence_detector is
27        port (seq : in std_logic;
28              enable : in std_logic;
29              reset : in std_logic;
30              clk : in std_logic;
31              cnt_1 : out std_logic_vector(2 downto 0);
32              cnt_2 : out std_logic_vector(2 downto 0));
33     end component;
34
35     component seven_segment_decoder is
36        port(code: in std_logic_vector (3 downto 0);
37             segments_out: out std_logic_vector (6 downto 0));
38     end component;
39
40     signal enable: std_logic;
41     signal R_reset: std_logic;
42     signal data_out: std_logic;
43     signal en : std_logic := '1'; -- enable for sequence_detector
44     signal count1, count2: std_logic_vector(2 downto 0);
45     signal code1, code2 : std_logic_vector(3 downto 0);
46
47  begin
48     clock_divider: Karen_Chen_clock_divider port map(en, reset, clk, enable);
49
50     R_reset <= not reset;
51
52     read_only_memory: ROM port map(enable, R_reset, data_out);
53
54     sequence_counter: Karen_Chen_sequence_detector port map(data_out, en, reset, enable, count1, count2);
55
56     code1 <= '0' & count1;
57     code2 <= '0' & count2;
58
59     decoder1: seven_segment_decoder port map(code1, HEX0);
60
61     decoder2: seven_segment_decoder port map(code2, HEX5);
62
63  end wrapper;
```

Figure 5. Wrapper behavioral VHDL description

```vhdl
library IEEE;
use IEEE.std_logic_1164.all;
use IEEE.NUMERIC_STD.all;

entity Karen_Chen_wrapper_tb is
--empty
end Karen_Chen_wrapper_tb;

architecture tb of Karen_Chen_wrapper_tb is

    component Karen_Chen_wrapper is
        port    ( reset   : in std_logic ;
                  clk     : in std_logic ;
                  HEX0    : out std_logic_vector(6 downto 0);
                  HEX5    : out std_logic_vector(6 downto 0));
    end component;

    signal reset: std_logic;
    signal clk:  std_logic;
    signal HEX0: std_logic_vector(6 downto 0);
    signal HEX5: std_logic_vector(6 downto 0);
    constant clock_period : time := 100 ms;
    --constant clock_period : time := 20 ns;

    begin
    uut: Karen_Chen_wrapper port map (reset, clk, HEX0, HEX5);

    clock_generation : process
    begin
        clk <= '1';
        wait for 50 ms;
        clk <= '0';
        wait for 50 ms;
    end process clock_generation ;

    process
        begin

        reset <= '0';
        wait for 700 ms;

        reset <= '1';
        wait;
    end process;
end tb;
```

Figure 6. Wrapper testbench code

The Wrapper circuit takes an asynchronous active-low reset signal (reset) and the clock signal (clk) as inputs. The outputs of this wrapper consist of two 7-segment vectors: HEX0 is designed to display the occurrence count of the pattern '1011,' while HEX5 is dedicated to presenting the occurrence count of the pattern '0010.' The implementation of the Wrapper involves several key components. First, a clock divider has been implemented to generate an enable signal at regular intervals, occurring every T clock cycles. This enable signal orchestrates the synchronized functioning of subsequent components. Additionally, a Read-Only Memory (ROM) circuit has been utilized to store the input bit sequence, with bits supplied at the positive edge of the clock signal. The core of the wrapper is a circuit comprising the clock divider, ROM, and a 7-segment decoder. This configuration ensures the accurate and periodic display of occurrence counts for both specified bit patterns at 1-second intervals.

3. Provide waveforms for each of the individual circuits (each section) and for the wrapper.

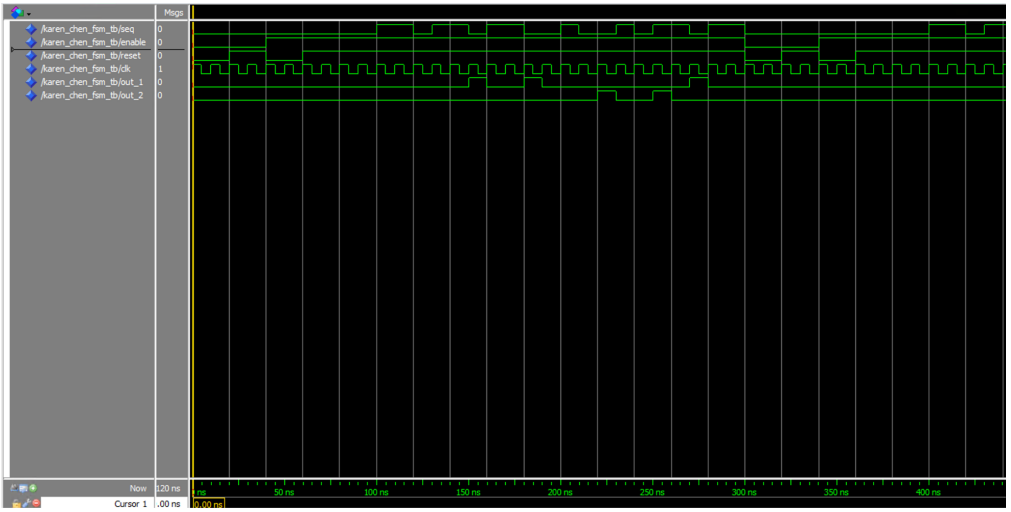

Figure 7. FSM simulation waveform (Zoom in version)

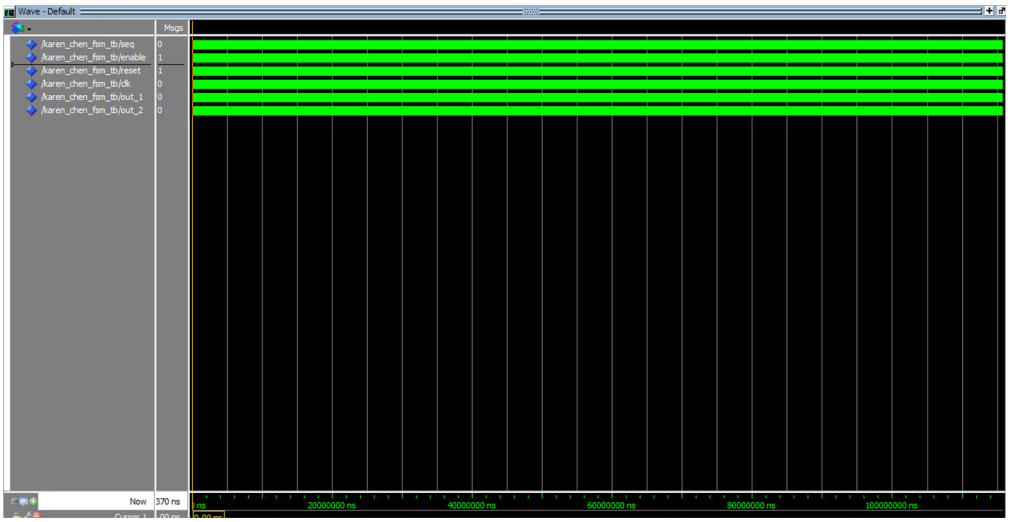

Figure 8. FSM simulation waveform (within a long time range)

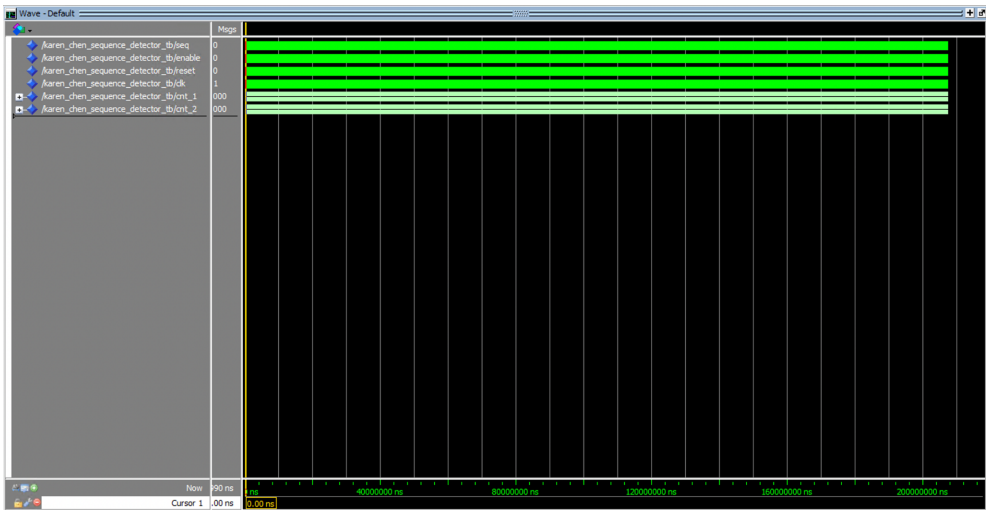

Figure 9. Sequence counter waveform (within a long time range)

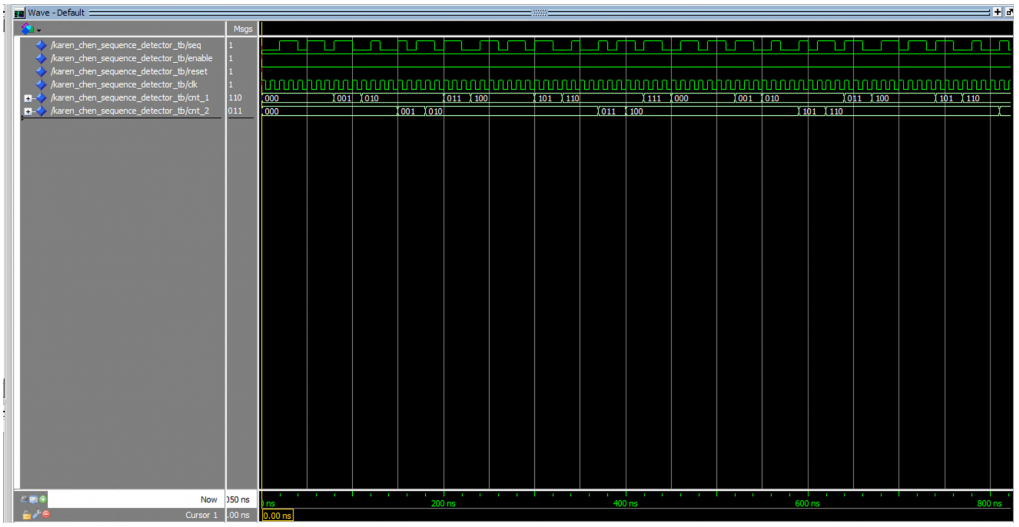

Figure 10. Sequence counter waveform (Zoom in version)

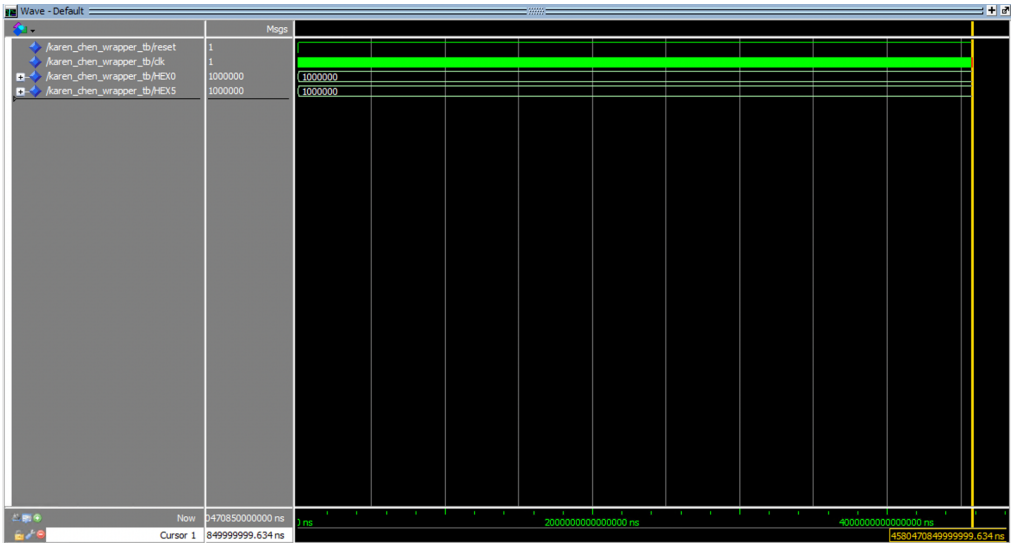

Figure 11 Wrapper simulation waveform (with a frequency of 50 MHz)

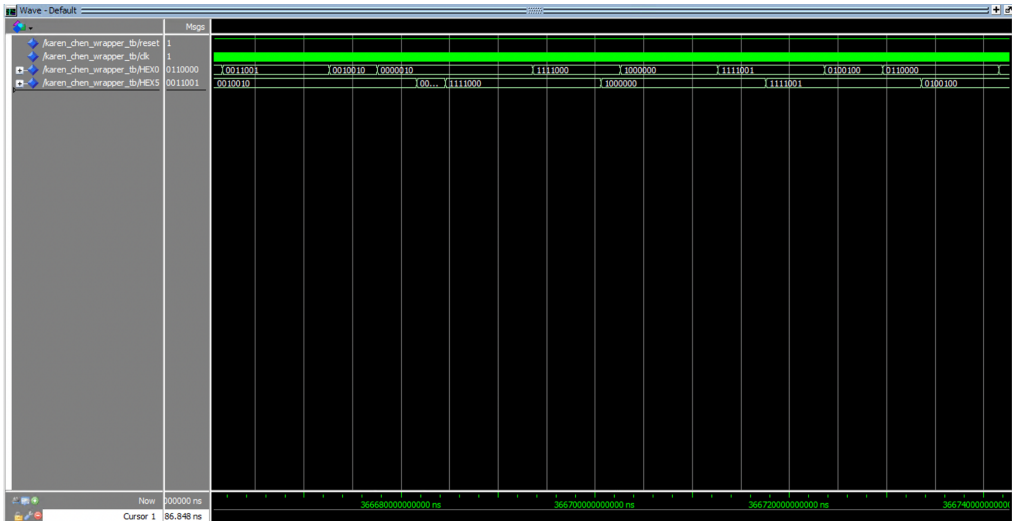

Figure 12. Wrapper simulation waveform (with a frequency of 10 Hz)

4. Perform timing analysis of the wrapper and find the critical path(s) of the circuit. What is the delay of the critical path(s)?

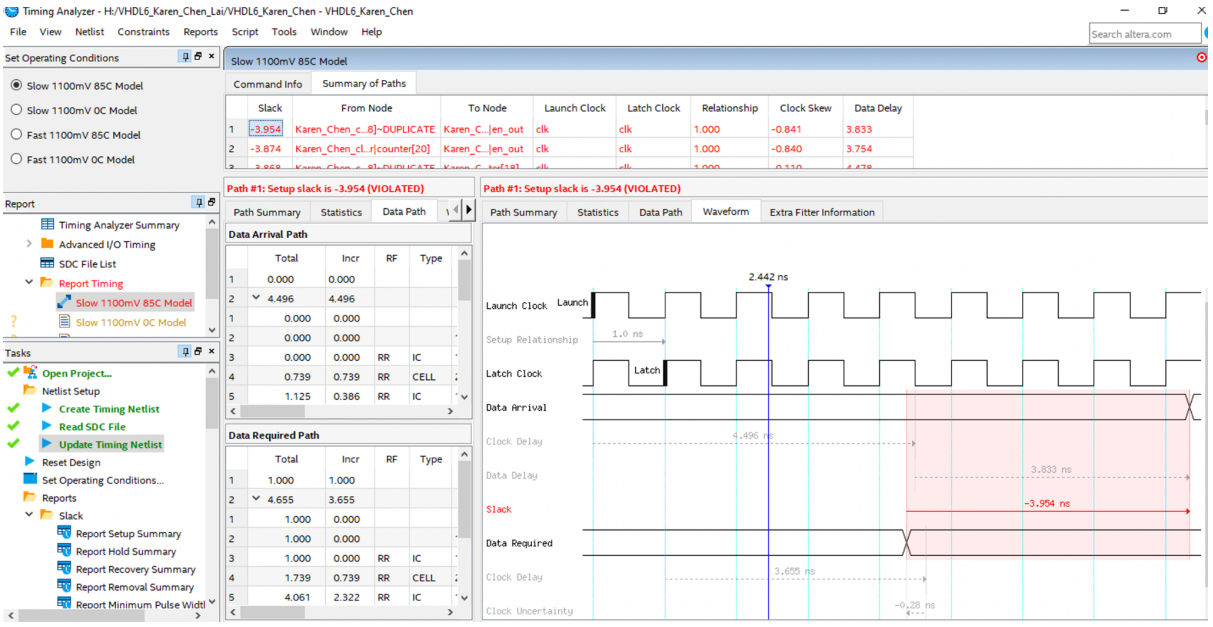

Figure 13. Timing Analysis of the wrapper with Slow 1100mV 85C model

	Slack	From	To	Recommendations
1	-3.954	Karen_Chen_clock_...er[18]~DUPLICATE	Karen_Chen_clock_...k_divider|en_out	Report recommendations for this path
2	-3.874	Karen_Chen_clock_...ider|counter[20]	Karen_Chen_clock_...k_divider|en_out	Report recommendations for this path
3	-3.868	Karen_Chen_clock_...er[18]~DUPLICATE	Karen_Chen_clock_...ider|counter[18]	Report recommendations for this path
4	-3.868	Karen_Chen_clock_...er[18]~DUPLICATE	Karen_Chen_clock_...ider|counter[10]	Report recommendations for this path
5	-3.867	Karen_Chen_clock_...er[18]~DUPLICATE	Karen_Chen_clock_...ider|counter[11]	Report recommendations for this path

Figure 14. Top critical paths of the wrapper

The critical path of the wrapper is shown in Fig 14. As Fig. 14 illustrates, the critical path has a data delay of 3.833 and a slack of -3.954.

5. Report the number of pins and logic modules used to fit your design on the FPGA board.

Flow Status	Successful - Tue Nov 28 00:03:46 2023
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Standard Edition
Revision Name	VHDL6_Karen_Chen
Top-level Entity Name	Karen_Chen_wrapper
Family	Cyclone V
Device	5CGXFC7C7F23C8
Timing Models	Final
Logic utilization (in ALMs)	55 / 56,480 (< 1 %)
Total registers	73
Total pins	16 / 268 (6 %)
Total virtual pins	0
Total block memory bits	0 / 7,024,640 (0 %)
Total DSP Blocks	0 / 156 (0 %)
Total HSSI RX PCSs	0 / 6 (0 %)
Total HSSI PMA RX Deserializers	0 / 6 (0 %)
Total HSSI TX PCSs	0 / 6 (0 %)
Total HSSI PMA TX Serializers	0 / 6 (0 %)
Total PLLs	0 / 13 (0 %)
Total DLLs	0 / 4 (0 %)

Figure 15. Compilation report of the wrapper

Number of pins used: 16/268 (6%)

Number of logic modules used: 55/56480 (< 1%)

Explanation of the Results

Two-FSM Approach:

The decision to use two Finite State Machines (FSMs) instead of a single one in the sequence detector implementation was driven by the need to efficiently handle two distinct bit patterns ('1011' and '0010'). Each FSM is dedicated to detecting one specific pattern. This approach simplifies the state transition logic and enhances the modularity of the design. Additionally, it facilitates easier debugging and maintenance.

VHDL Code Implementation:

The VHDL code for the sequence detector and counter circuits follows a modular and structured design. Each entity is defined with specific input and output ports, adhering to the provided guidelines. The state diagrams formed the basis for the code, ensuring a clear mapping of states, transitions, and output conditions. The asynchronous reset and enable signals were integrated to control the state transitions and ensure correct operation.

Waveform Analysis:

The waveform diagrams for each individual circuit section and the wrapper provide a visual representation of the circuit's behavior. The waveforms illustrate the transitions between different states, the activation of enable signals, and the detection of the specified bit patterns. The correctness of the design can be verified by examining the outputs ('out_1' and 'out_2') in response to the input bit sequence. For all the simulation waveforms we are putting two images in order to better visualize the proper changes of output after certain time range. For the wrapper, we tried it with a frequency of 10 Hz instead of 50 MHz, as

shown in Fig.12, in which we can see that all the decoded outputs are increasing starting from 0 to 7, then restart again from 0, where demonstrates that the wrapper is working properly.

Timing Analysis and Critical Path:

Timing analysis of the wrapper involves assessing the critical path(s) in the circuit. The critical path is the longest path that determines the maximum delay through the design. By identifying the critical path, designers can optimize the circuit for better performance. The delay of the critical path(s) is a crucial metric, as it influences the overall speed and reliability of the implemented circuit.

Conclusions

Through this experiment, we learned how to:

- Implement the Moore type Finite State Machine and learnt the advantage of utilizing two FSMs for detecting distinct bit patterns in sequence
- Reuse the other circuits implemented in the past assignment, such as counter, clock divider, and seven segment decoder to wrap them together
- Enhance our understanding on waveform analysis through changing the clock period.
- Integrate the circuit into Altera DE1-SoC FPGA board through assigning the inputs towards correct pins.